

Advanced Machine Learning

Lecture 2: Finding the Network Parameters

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

22 October 2024



Outline

Repetition

- Support vector machines (SVM)
- Activation functions
- Artificial neural networks (ANN)

Basics

- Universal approximation theorem
- Barron spaces
- Optimization
- Stochastic gradient descent
- Training by backpropagation

Outline

Repetition

- Support vector machines (SVM)
- Activation functions
- Artificial neural networks (ANN)

Basics

- Universal approximation theorem
- Barron spaces
- Optimization
- Stochastic gradient descent
- Training by backpropagation

SVM – support vector machine

Example: Boolean functions $f : \{0, 1\}^2 \rightarrow \{0, 1\}$

(0, 1) (1, 1)

(0, 0) (1, 0)

logical OR

(0, 1) (1, 1)

(0, 0) (1, 0)

logical AND

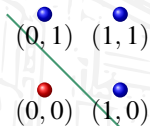
(0, 1) (1, 1)

(0, 0) (1, 0)

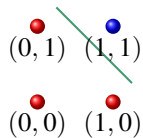
logical XOR

SVM – support vector machine

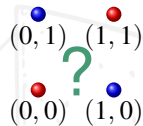
Example: Boolean functions $f : \{0, 1\}^2 \rightarrow \{0, 1\}$



logical OR



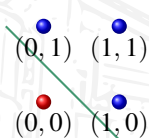
logical AND



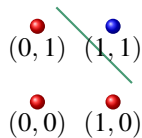
logical XOR

SVM – support vector machine

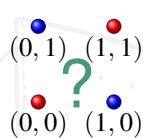
Example: Boolean functions $f : \{0, 1\}^2 \rightarrow \{0, 1\}$



logical OR



logical AND



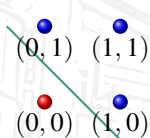
logical XOR

Maximal margin SVM (separating hyperplane $\mathbf{w}^T \mathbf{x} + b = 0$):

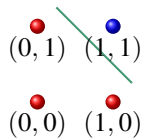
$$2x_1 + 2x_2 - 1 = 0, \quad 2x_1 + 2x_2 - 3 = 0, \quad (\text{does not exist})$$

SVM – support vector machine

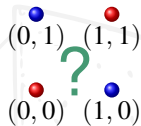
Example: Boolean functions $f : \{0, 1\}^2 \rightarrow \{0, 1\}$



logical OR



logical AND



logical XOR

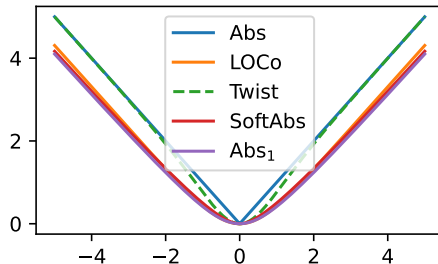
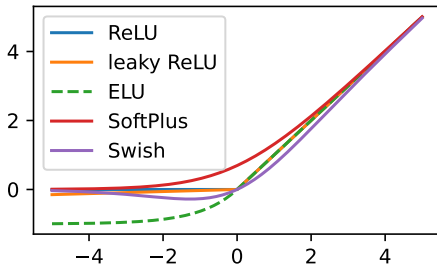
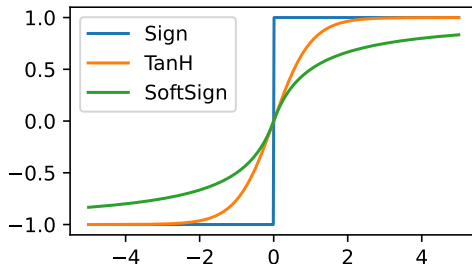
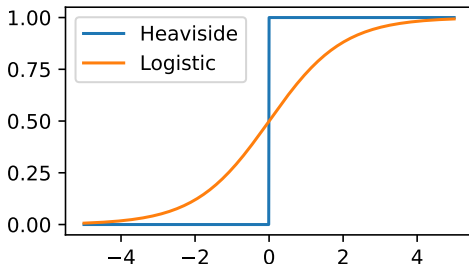
Maximal margin SVM (separating hyperplane $\mathbf{w}^T \mathbf{x} + b = 0$):

$$2x_1 + 2x_2 - 1 = 0, \quad 2x_1 + 2x_2 - 3 = 0, \quad (\text{does not exist})$$

The two existing **SVM** are **solutions** of

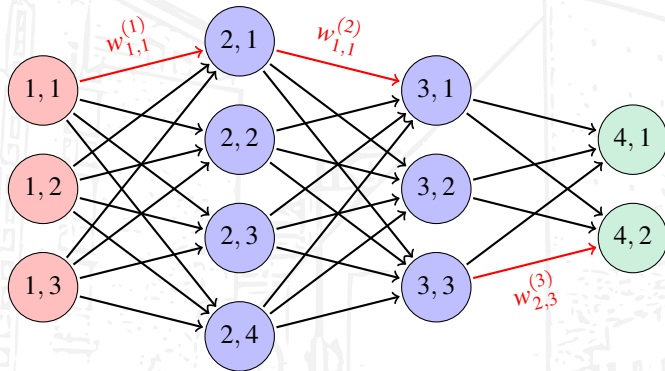
$$\begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ b \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}, \quad \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ b \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}.$$

Activation functions



SVM + activation functions = ANN

Artificial neural network (ANN): columnwise combination of SVM + componentwise application of an activation function.



“Feedforward”:

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m.$$

Outline

Repetition

- Support vector machines (SVM)
- Activation functions
- Artificial neural networks (ANN)

Basics

- Universal approximation theorem
- Barron spaces
- Optimization
- Stochastic gradient descent
- Training by backpropagation

ANN are functions ... but which?

Which functions $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ can be approximated by an ANN, and if so, how well?

Transferred to **approximability of scalar** $f : \mathbb{R}^n \rightarrow \mathbb{R}$, preliminary work:

- ▶ Kolmogorov [9, 10]
- ▶ Sprecher [17]

Result: every continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ of **several** variables is a finite linear combination of **univariate** functions.

Pros and Cons:

- ▶ functions **continuous**
- ▶ ANN w/ (at least) **one hidden layer** of $2n + 1$ neurons
- ▶ functions **unknown activation functions**
- ▶ functions per layer **different, not known in advance**

First steps

End of 80s: first publications concerning more general activation functions, according to Hornik, Stinchcombe, and White [8],

- ▶ Cun [1] (Dissertation of Yann Le Cun)
- ▶ Lapedes and Farber [12]

show that w/ **two hidden layers** arbitrary continuous functions can be approximated if the **activation function is monotone and sigmoidal**.

First steps

End of 80s: first publications concerning more general activation functions, according to Hornik, Stinchcombe, and White [8],

- ▶ Cun [1] (Dissertation of Yann Le Cun)
- ▶ Lapedes and Farber [12]

show that w/ **two hidden layers** arbitrary continuous functions can be approximated if the **activation function is monotone and sigmoidal**.

A function $\sigma : \mathbb{R} \rightarrow [0, 1]$ is **sigmoidal**, if

$$\lim_{x \rightarrow -\infty} \sigma(x) = 0, \quad \lim_{x \rightarrow \infty} \sigma(x) = 1.$$

First steps

End of 80s: first publications concerning more general activation functions, according to Hornik, Stinchcombe, and White [8],

- ▶ Cun [1] (Dissertation of Yann Le Cun)
- ▶ Lapedes and Farber [12]

show that w/ **two hidden layers** arbitrary continuous functions can be approximated if the **activation function is monotone and sigmoidal**.

A function $\sigma : \mathbb{R} \rightarrow [0, 1]$ is **sigmoidal**, if

$$\lim_{x \rightarrow -\infty} \sigma(x) = 0, \quad \lim_{x \rightarrow \infty} \sigma(x) = 1.$$

Rationale in Lapedes and Farber [12] rather **informal**, locally bounded “bumps” are constructed and the function is assembled using these.

Cybenko's universal approximation theorem (UAT)

First **mathematically rigorous** proof published by Cybenko [2] in 1989:

- ▶ every continuous function $\mathbf{f} : I_n := [0, 1]^n \rightarrow \mathbb{R}$ can be approximated arbitrarily well using **one** hidden layer and a **discriminatory** activation function
- ▶ activation function $\sigma : \mathbb{R} \rightarrow [0, 1]$ **discriminatory**: for all measures μ

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + b) d\mu(x) = 0 \quad \forall \mathbf{w} \in \mathbb{R}^n, \forall b \in \mathbb{R}$$

implies $\mu = 0$

- ▶ all continuous sigmoidal activation functions are discriminatory, e.g., Logistic_k

Cybenko's universal approximation theorem (UAT)

First **mathematically rigorous** proof published by Cybenko [2] in 1989:

- ▶ every continuous function $\mathbf{f} : I_n := [0, 1]^n \rightarrow \mathbb{R}$ can be approximated arbitrarily well using **one** hidden layer and a **discriminatory** activation function
- ▶ activation function $\sigma : \mathbb{R} \rightarrow [0, 1]$ **discriminatory**: for all measures μ

$$\int_{I_n} \sigma(\mathbf{w}^T \mathbf{x} + b) d\mu(x) = 0 \quad \forall \mathbf{w} \in \mathbb{R}^n, \forall b \in \mathbb{R}$$

implies $\mu = 0$

- ▶ all continuous sigmoidal activation functions are discriminatory, e.g., Logistic_k

Other works with similar results appeared in 1989 in the journal “**Neural Networks**”:

- ▶ Funahashi [5]
- ▶ Hornik, Stinchcombe, and White [8]

Hornik's universal approximation theorem (UAT)

Hornik [7, Theorem 1–4]:

($\rightsquigarrow$ “Numerical Mathematics II”, summer semester)

Theorem (Universal approximation theorem)

Let $N(\sigma)$ denote the set of all possible functions of an ANN with one hidden layer of n neurons.

- 1 If σ is bounded and nonconstant, then $N(\sigma)$ is dense in $L^p(\mu)$ for all finite measures μ on $\mathbb{R}^n$ (i.e., those with $\|1\|_{\mu,p} < \infty$).
- 2 If σ moreover is continuous, then $N(\sigma)$ is dense in all $C(K)$, $K \subset \mathbb{R}^n$ compact.
- 3 If $\sigma \in C^m(\mathbb{R}^n)$ is bounded and nonconstant, then $N(\sigma)$ is uniformly m -dense on compacta in $C^m(\mathbb{R}^n)$ and dense in $C^{m,p}(\mu)$ for all finite measures μ with compact support.
- 4 If $\sigma \in C^m(\mathbb{R}^n)$ is nonconstant and all its derivatives up to order m are bounded, then $N(\sigma)$ is dense in $C^{m,p}(\mu)$ for all finite measures μ on $\mathbb{R}^n$.

Based on Cybenko's proof. Hornik [7, Theorem 5]: σ discriminatory if bounded and nonconstant.
 $C^{m,p}(\mu) := \{f \in C^m(\mathbb{R}^n) \mid \sum_{|\alpha| \leq m} \int |D^\alpha f|^p d\mu < \infty\}$ weighted Sobolev space.

UAT: extensions

Results thus far: mostly **one hidden layer** with **unspecified number of neurons**, **restrictions on activation functions**.

UAT: extensions

Results thus far: mostly **one hidden layer** with **unspecified number of neurons**, **restrictions on activation functions**.

Extensions:

UAT: extensions

Results thus far: mostly **one hidden layer** with **unspecified number of neurons**, **restrictions on activation functions**.

Extensions:

- Kůrková [11]: **number of necessary neurons**. Derived by approximation of the functions in Kolmogorov' Theorem using sigmoidal functions. For every continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ and $\epsilon > 0$ exists ANN with two layers that approximates to order ϵ , where the first layer contains $O(n^3)$ and the second layer $O(n^{n+1})$ neurons, i.e., **exponentially** many

UAT: extensions

Results thus far: mostly **one hidden layer** with **unspecified number of neurons**, **restrictions on activation functions**.

Extensions:

- ▶ Kůrková [11]: **number of necessary neurons**. Derived by approximation of the functions in Kolmogorov' Theorem using sigmoidal functions. For every continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ and $\epsilon > 0$ exists ANN with two layers that approximates to order ϵ , where the first layer contains $O(n^3)$ and the second layer $O(n^{n+1})$ neurons, i.e., **exponentially** many
- ▶ Leshno et al. [13], see also Pinkus [16]: theoretically **every non-polynomial activation function** can be used to approximate a continuous function on compacta to any order

UAT: extensions

Results thus far: mostly **one hidden layer** with **unspecified number of neurons**, **restrictions on activation functions**.

Extensions:

- ▶ Kůrková [11]: **number of necessary neurons**. Derived by approximation of the functions in Kolmogorov' Theorem using sigmoidal functions. For every continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ and $\epsilon > 0$ exists ANN with two layers that approximates to order ϵ , where the first layer contains $O(n^3)$ and the second layer $O(n^{n+1})$ neurons, i.e., **exponentially** many
- ▶ Leshno et al. [13], see also Pinkus [16]: theoretically **every non-polynomial activation function** can be used to approximate a continuous function on compacta to any order
- ▶ Park and Sandberg [15]: UAT for **RBF-ANN** (RBF: radial basis function $\rightsquigarrow$ "Numerical Mathematics II", summer semester)

UAT: descriptive & deep

Descriptive: for two hidden layers, Logistic, and $n = 1$ as well as $n = 2$ in Nielsen [14, Chapter 4: A visual proof that neural nets can compute any function]; can already be found in simplified form in Lapedes and Farber [12].

UAT: descriptive & deep

Descriptive: for two hidden layers, Logistic, and $n = 1$ as well as $n = 2$ in Nielsen [14, Chapter 4: A visual proof that neural nets can compute any function]; can already be found in simplified form in Lapedes and Farber [12].

In the **exercises** we analyse the case of one hidden layer, $n = 1$, and ReLU. This case reduces the corresponding ANN to the computation of an interpolating or approximating piecewise linear spline (splines: “Numerical Mathematics II”, summer semester).

UAT: descriptive & deep

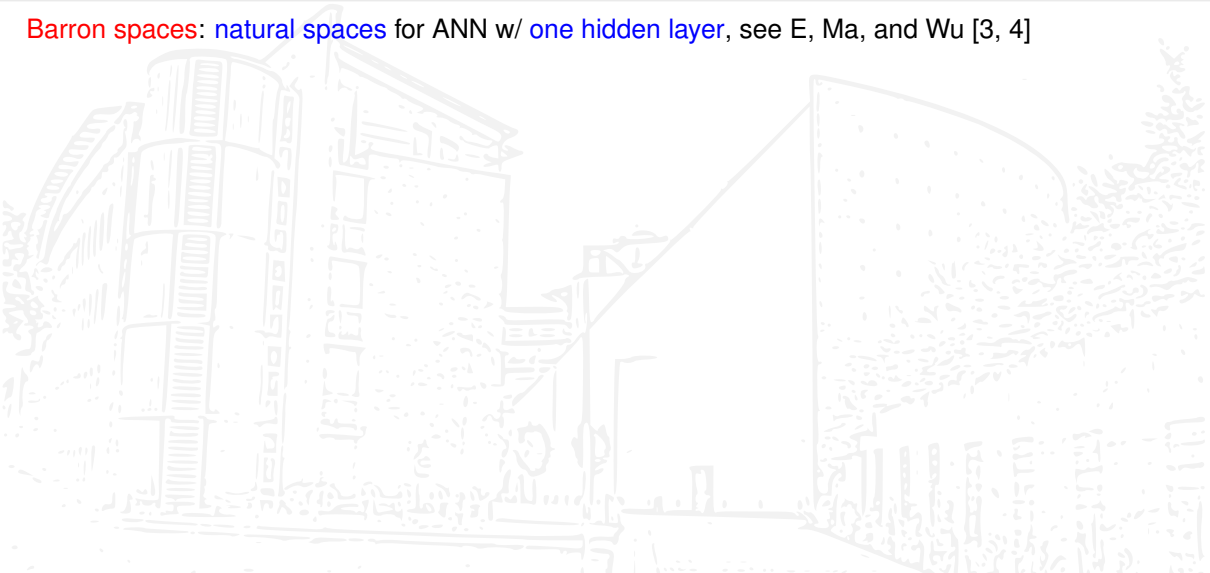
Descriptive: for two hidden layers, Logistic, and $n = 1$ as well as $n = 2$ in Nielsen [14, Chapter 4: A visual proof that neural nets can compute any function]; can already be found in simplified form in Lapedes and Farber [12].

In the **exercises** we analyse the case of one hidden layer, $n = 1$, and ReLU. This case reduces the corresponding ANN to the computation of an interpolating or approximating piecewise linear spline (splines: “Numerical Mathematics II”, summer semester).

Recent research: deep networks with bounded number of neurons per layer. There are studies that show that by using more layers with less neurons better results can be obtained than the other way round. Hanin [6], e.g., derives bound on the depth of the network based on the properties of the function to be approximated, where the neurons per layer are bounded by $n + 1$.

Barron spaces

Barron spaces: natural spaces for ANN w/ one hidden layer, see E, Ma, and Wu [3, 4]



Barron spaces

Barron spaces: natural spaces for ANN w/ one hidden layer, see E, Ma, and Wu [3, 4]

Set of functions (activation function $a \in \{\text{ReLU}, \text{leaky ReLU}, \text{Abs}\}$):

$$\left\{ f \mid f(\mathbf{x}) = \int_{\mathbb{R}^1 \times \mathbb{R}^d \times \mathbb{R}^1} c a(\mathbf{w}^\top \mathbf{x} + b) \rho(dc, d\mathbf{w}, db), \quad \rho \text{ probability distribution} \right\}$$

Barron spaces

Barron spaces: natural spaces for ANN w/ one hidden layer, see E, Ma, and Wu [3, 4]

Set of functions (activation function $a \in \{\text{ReLU}, \text{leaky ReLU}, \text{Abs}\}$):

$$\left\{ f \mid f(\mathbf{x}) = \int_{\mathbb{R}^1 \times \mathbb{R}^d \times \mathbb{R}^1} c a(\mathbf{w}^\top \mathbf{x} + b) \rho(dc, d\mathbf{w}, db), \quad \rho \text{ probability distribution} \right\}$$

Barron norms:

$$\|f\|_{\mathcal{B}_p} := \inf_{\rho} (\mathbb{E}_{\rho} (|c|^p (\|\mathbf{w}\|_1 + |b|)^p))^{1/p}, \quad 1 \leq p \leq \infty$$

Barron space = functions from set w/ **finite Barron norm**

Barron spaces

Barron spaces: natural spaces for ANN w/ one hidden layer, see E, Ma, and Wu [3, 4]

Set of functions (activation function $a \in \{\text{ReLU}, \text{leaky ReLU}, \text{Abs}\}$):

$$\left\{ f \mid f(\mathbf{x}) = \int_{\mathbb{R}^1 \times \mathbb{R}^d \times \mathbb{R}^1} c a(\mathbf{w}^\top \mathbf{x} + b) \rho(dc, d\mathbf{w}, db), \quad \rho \text{ probability distribution} \right\}$$

Barron norms:

$$\|f\|_{\mathcal{B}_p} := \inf_{\rho} (\mathbb{E}_{\rho} (|c|^p (\|\mathbf{w}\|_1 + |b|)^p))^{1/p}, \quad 1 \leq p \leq \infty$$

Barron space = functions from set w/ **finite Barron norm**

For ReLU: Barron space given by $\mathcal{B} := \mathcal{B}_1 = \dots = \mathcal{B}_{\infty}$

Barron spaces

Barron spaces: natural spaces for ANN w/ one hidden layer, see E, Ma, and Wu [3, 4]

Set of functions (activation function $a \in \{\text{ReLU}, \text{leaky ReLU}, \text{Abs}\}$):

$$\left\{ f \mid f(\mathbf{x}) = \int_{\mathbb{R}^1 \times \mathbb{R}^d \times \mathbb{R}^1} c a(\mathbf{w}^\top \mathbf{x} + b) \rho(dc, d\mathbf{w}, db), \quad \rho \text{ probability distribution} \right\}$$

Barron norms:

$$\|f\|_{\mathcal{B}_p} := \inf_{\rho} (\mathbb{E}_{\rho} (|c|^p (\|\mathbf{w}\|_1 + |b|)^p))^{1/p}, \quad 1 \leq p \leq \infty$$

Barron space = functions from set w/ **finite Barron norm**

For ReLU: Barron space given by $\mathcal{B} := \mathcal{B}_1 = \dots = \mathcal{B}_{\infty}$

Beating the **curse of dimensionality** (m neurons in hidden layer, $\mathbf{p}$ vector of parameters):

$$\exists f_m = f_m(\mathbf{p}) \quad \text{such that} \quad \|f - f_m\|^2 \leq \frac{3\|f\|_{\mathcal{B}}^2}{m}, \quad \|\mathbf{p}\|_{\mathcal{P}} := \frac{1}{m} \sum_{j=1}^m |c_j| (\|\mathbf{w}_j\|_1 + |b_j|) \leq 2\|f\|_{\mathcal{B}}$$

Rademacher complexity & generalization

Rademacher complexity ($\mathcal{F}$ set of functions, $\mathbf{x}_1, \dots, \mathbf{x}_n$ samples, $P(\xi = 1) = P(\xi = -1) = .5$):

$$\text{Rad}_n(\mathcal{F}) := \frac{1}{n} \mathbb{E}_{\xi} \sup_{f \in \mathcal{F}} \sum_{i=1}^n \xi_i f(\mathbf{x}_i)$$

Rademacher complexity & generalization

Rademacher complexity ($\mathcal{F}$ set of functions, $\mathbf{x}_1, \dots, \mathbf{x}_n$ samples, $P(\xi = 1) = P(\xi = -1) = .5$):

$$\text{Rad}_n(\mathcal{F}) := \frac{1}{n} \mathbb{E}_{\xi} \sup_{f \in \mathcal{F}} \sum_{i=1}^n \xi_i f(\mathbf{x}_i)$$

Rademacher complexity of **Barron space**:

$$\mathcal{F}_Q := \{f \in \mathcal{B} \mid \|f\|_{\mathcal{B}} \leq Q\} \Rightarrow \text{Rad}_n(\mathcal{F}_Q) \leq 2Q \sqrt{\frac{2 \ln(2d)}{n}}$$

Rademacher complexity & generalization

Rademacher complexity ($\mathcal{F}$ set of functions, $\mathbf{x}_1, \dots, \mathbf{x}_n$ samples, $P(\xi = 1) = P(\xi = -1) = .5$):

$$\text{Rad}_n(\mathcal{F}) := \frac{1}{n} \mathbb{E}_{\xi} \sup_{f \in \mathcal{F}} \sum_{i=1}^n \xi_i f(\mathbf{x}_i)$$

Rademacher complexity of **Barron space**:

$$\mathcal{F}_Q := \{f \in \mathcal{B} \mid \|f\|_{\mathcal{B}} \leq Q\} \Rightarrow \text{Rad}_n(\mathcal{F}_Q) \leq 2Q \sqrt{\frac{2 \ln(2d)}{n}}$$

Barron space has **low Rademacher complexity**:

- ▶ functions in Barron spaces can be **learned efficiently**
- ▶ **generalization error** with probability $1 - \delta$ behaves like (multiple of)

$$\text{Rad}_n(\mathcal{F}) + \sqrt{\frac{\ln(1/\delta)}{n}}$$

ANN as function approximators

ANN

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m,$$

defines **function** $\mathbf{f}$ expressed in terms of **parameters** $\mathbf{W}_i, \mathbf{b}_i, i = 1, \dots, m-1$.

ANN as function approximators

ANN

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := \sigma(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m,$$

defines **function** $\mathbf{f}$ expressed in terms of **parameters** $\mathbf{W}_i, \mathbf{b}_i, i = 1, \dots, m-1$.

Simplified presentation: **vector of all parameters** $\mathbf{p}$. Goal: find parameters $\mathbf{p}$ such that

$$\mathbf{f}(\mathbf{x}_j) \approx \mathbf{y}_j, \quad j = 1, \dots, N.$$

ANN as function approximators

ANN

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := \sigma(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m,$$

defines **function** $\mathbf{f}$ expressed in terms of **parameters** $\mathbf{W}_i, \mathbf{b}_i, i = 1, \dots, m-1$.

Simplified presentation: **vector of all parameters** $\mathbf{p}$. Goal: find parameters $\mathbf{p}$ such that

$$\mathbf{f}(\mathbf{x}_j) \approx \mathbf{y}_j, \quad j = 1, \dots, N.$$

Choose **cost (or loss) function** to be minimized, e.g., mean squared error (MSE) (here scaled):

$$C(\mathbf{p}) := \frac{1}{2N} \sum_{j=1}^N \|\mathbf{f}(\mathbf{x}_j) - \mathbf{y}_j\|_2^2.$$

ANN as function approximators

ANN

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := \mathbf{a}_i(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m,$$

defines **function** $\mathbf{f}$ expressed in terms of **parameters** $\mathbf{W}_i, \mathbf{b}_i, i = 1, \dots, m-1$.

Simplified presentation: **vector of all parameters** $\mathbf{p}$. Goal: find parameters $\mathbf{p}$ such that

$$\mathbf{f}(\mathbf{x}_j) \approx \mathbf{y}_j, \quad j = 1, \dots, N.$$

Choose **cost (or loss) function** to be minimized, e.g., mean squared error (MSE) (here scaled):

$$C(\mathbf{p}) := \frac{1}{2N} \sum_{j=1}^N \|\mathbf{f}(\mathbf{x}_j) - \mathbf{y}_j\|_2^2.$$

Important that C splits into **differentiable similar parts** $\rightsquigarrow$ **optimization**:

$$C_j(\mathbf{p}) := \frac{1}{2} \|\mathbf{f}(\mathbf{x}_j) - \mathbf{y}_j\|_2^2, \quad C(\mathbf{p}) = \frac{1}{N} \sum_{j=1}^N C_j(\mathbf{p}).$$

1D optimization

Digression: minima of the differentiable function

$$f(x) = \left(1 + \frac{x^2}{10}\right) \sin(3x) + \frac{x^2}{9} \quad (C \rightsquigarrow f, \quad \mathbf{p} \rightsquigarrow x)$$

in the interval $[-5, 5]$.

1D optimization

Digression: minima of the differentiable function

$$f(x) = \left(1 + \frac{x^2}{10}\right) \sin(3x) + \frac{x^2}{9} \quad (C \rightsquigarrow f, \quad \mathbf{p} \rightsquigarrow x)$$

in the interval $[-5, 5]$.

In this interval:

- ▶ five **local minima**
- ▶ the one closest to zero is the **global minimum**

1D optimization

Digression: minima of the differentiable function

$$f(x) = \left(1 + \frac{x^2}{10}\right) \sin(3x) + \frac{x^2}{9} \quad (C \rightsquigarrow f, \quad \mathbf{p} \rightsquigarrow x)$$

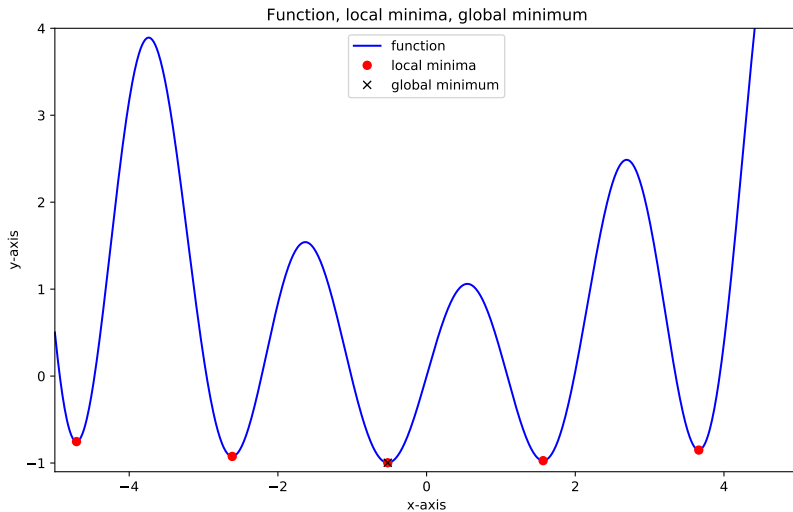
in the interval $[-5, 5]$.

In this interval:

- ▶ five **local minima**
- ▶ the one closest to zero is the **global minimum**

A plot of the function, the local minima, and the global minimum in the interval $[-5, 5]$ can be found on the next page.

1D optimization



1D optimization

Function

$$f(x) = \left(1 + \frac{x^2}{10}\right) \sin(3x) + \frac{x^2}{9}$$

analytical $\rightsquigarrow$ infinitely often differentiable.

1D optimization

Function

$$f(x) = \left(1 + \frac{x^2}{10}\right) \sin(3x) + \frac{x^2}{9}$$

analytical $\rightsquigarrow$ infinitely often **differentiable**.

- ▶ **extrema** $\rightsquigarrow$ roots of **first derivative**
- ▶ **second derivative** in **extrema** here nonzero
- ▶ sign of **second derivative** determines **minimum**/maximum

1D optimization

Function

$$f(x) = \left(1 + \frac{x^2}{10}\right) \sin(3x) + \frac{x^2}{9}$$

analytical $\rightsquigarrow$ infinitely often **differentiable**.

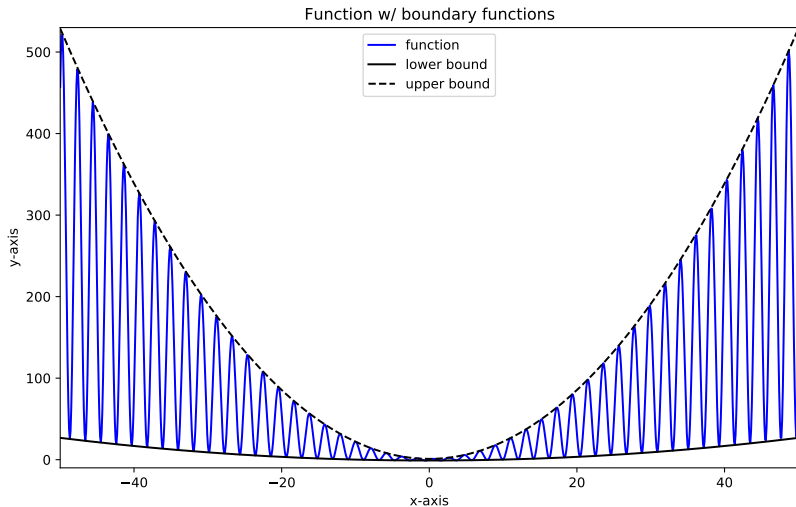
- ▶ **extrema** $\rightsquigarrow$ roots of **first derivative**
- ▶ **second derivative** in **extrema** here nonzero
- ▶ sign of **second derivative** determines **minimum**/maximum

Global minimum? **Global bounds** using $|\sin(x)| \leq 1$:

$$\ell(x) := \frac{x^2}{9} - \frac{x^2}{10} - 1 = \frac{x^2}{90} - 1 \leq f(x),$$

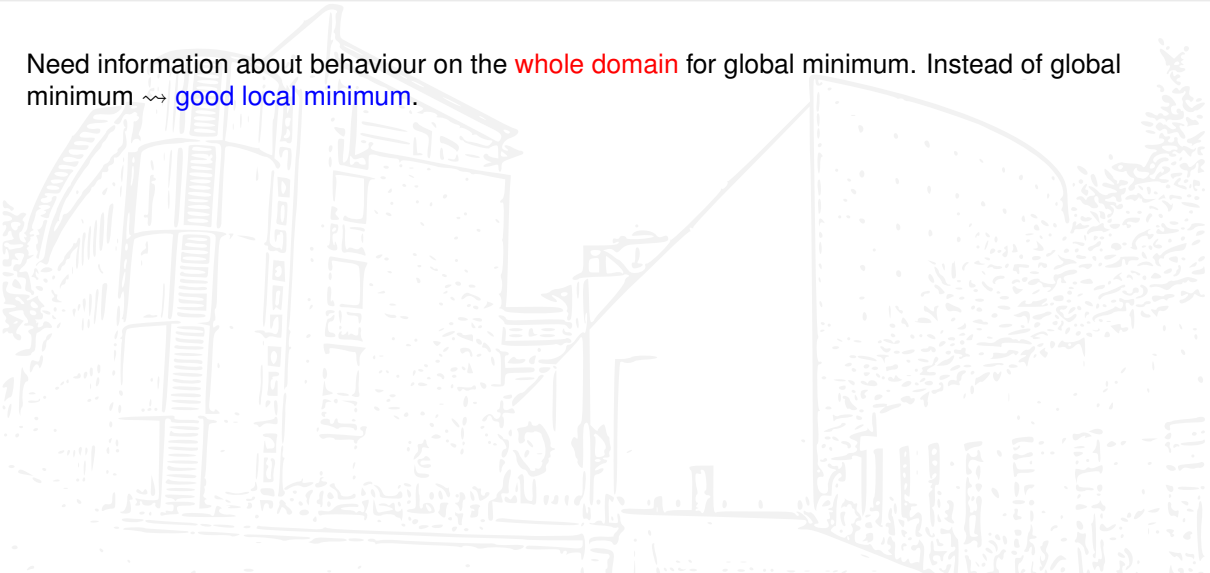
$$u(x) := \frac{x^2}{9} + \frac{x^2}{10} + 1 = \frac{19x^2}{90} + 1 \geq f(x).$$

1D optimization



1D optimization

Need information about behaviour on the **whole domain** for global minimum. Instead of global minimum $\rightsquigarrow$ **good local minimum**.



1D optimization

Need information about behaviour on the **whole domain** for global minimum. Instead of global minimum $\rightsquigarrow$ **good local minimum**.

Here: **knowledge of borders** gives good approximations for minima as $-\pi/6 + 2k\pi/3$, $k \in \mathbb{Z}$. Owing to the location of these and the curvature of the lower bound function local minimum global, which lies closest to zero $\rightsquigarrow$ close to $-\pi/6$.

1D optimization

Need information about behaviour on the **whole domain** for global minimum. Instead of global minimum $\rightsquigarrow$ **good local minimum**.

Here: **knowledge of borders** gives good approximations for minima as $-\pi/6 + 2k\pi/3$, $k \in \mathbb{Z}$. Owing to the location of these and the curvature of the lower bound function local minimum global, which lies closest to zero $\rightsquigarrow$ close to $-\pi/6$.

(Local) minima mostly not computable in closed form $\rightsquigarrow$ numerical approximations:

- ▶ **method of steepest descent**: plays dominant role in neural networks
- ▶ **Newton's method**: only plays a secondary role in neural networks ($\rightsquigarrow$ K-FAC, Shampoo, ...)

1D optimization

Need information about behaviour on the **whole domain** for global minimum. Instead of global minimum $\rightsquigarrow$ **good local minimum**.

Here: **knowledge of borders** gives good approximations for minima as $-\pi/6 + 2k\pi/3$, $k \in \mathbb{Z}$. Owing to the location of these and the curvature of the lower bound function local minimum global, which lies closest to zero $\rightsquigarrow$ close to $-\pi/6$.

(Local) minima mostly not computable in closed form $\rightsquigarrow$ numerical approximations:

- ▶ **method of steepest descent**: plays dominant role in neural networks
- ▶ **Newton's method**: only plays a secondary role in neural networks ($\rightsquigarrow$ K-FAC, Shampoo, ...)

Method of steepest descent also termed **gradient descent** (GD).

Method of steepest descent

Consider first $f : \mathbb{R} \rightarrow \mathbb{R}$. Based on **derivatives** $\rightsquigarrow$ require (piecewise) $f \in C^1$.



Method of steepest descent

Consider first $f : \mathbb{R} \rightarrow \mathbb{R}$. Based on **derivatives** $\rightsquigarrow$ require (piecewise) $f \in C^1$.

Derivative in x is **slope**:

- ▶ **derivative positive** $\rightsquigarrow$ proceed in negative direction
- ▶ **derivative negative** $\rightsquigarrow$ proceed in positive direction
- ▶ **derivative continuous, nonzero** $\rightsquigarrow$ there exists $\delta > 0$ such that for $\delta > \alpha > 0$

$$f(x - f'(x)\alpha) < f(x)$$

Method of steepest descent

Consider first $f : \mathbb{R} \rightarrow \mathbb{R}$. Based on **derivatives** $\rightsquigarrow$ require (piecewise) $f \in C^1$.

Derivative in x is **slope**:

- ▶ **derivative positive** $\rightsquigarrow$ proceed in negative direction
- ▶ **derivative negative** $\rightsquigarrow$ proceed in positive direction
- ▶ **derivative continuous, nonzero** $\rightsquigarrow$ there exists $\delta > 0$ such that for $\delta > \alpha > 0$

$$f(x - f'(x)\alpha) < f(x)$$

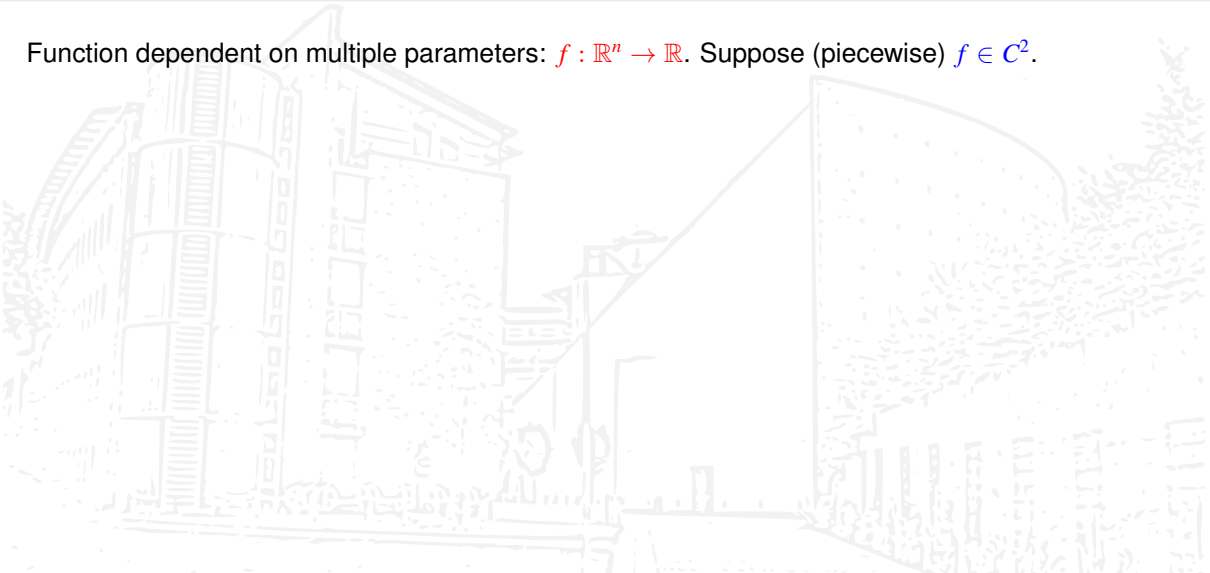
Iteration: choose appropriate $x_0 \in \mathbb{R}$, iterate

$$x_{k+1} := x_k - f'(x_k)\alpha_k, \quad k \in \mathbb{N}_0,$$

where parameter $\alpha_k > 0$ is appropriate **step length**.

Method of steepest descent

Function dependent on multiple parameters: $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Suppose (piecewise) $f \in C^2$.



Method of steepest descent

Function dependent on multiple parameters: $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Suppose (piecewise) $f \in C^2$.

Derivative vs. gradient: (for the standard skalar product $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y}$)

- ▶ **derivative** $Df : \mathbb{R}^n \rightarrow \mathbb{R}$ is linear map $\rightsquigarrow$ **row vector**
- ▶ **gradient** ∇f is transposed derivative $\rightsquigarrow$ **column vector**

$$Df_{\mathbf{x}} := \left(\frac{\partial f}{\partial x_1}(\mathbf{x}) \quad \cdots \quad \frac{\partial f}{\partial x_n}(\mathbf{x}) \right), \quad \nabla f(\mathbf{x}) := \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix} = Df_{\mathbf{x}}^\top.$$

Method of steepest descent

Function dependent on multiple parameters: $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Suppose (piecewise) $f \in C^2$.

Derivative vs. gradient: (for the standard skalar product $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y}$)

- ▶ **derivative** $Df : \mathbb{R}^n \rightarrow \mathbb{R}$ is linear map $\rightsquigarrow$ **row vector**
- ▶ **gradient** ∇f is transposed derivative $\rightsquigarrow$ **column vector**

$$Df_{\mathbf{x}} := \left(\frac{\partial f}{\partial x_1}(\mathbf{x}) \quad \cdots \quad \frac{\partial f}{\partial x_n}(\mathbf{x}) \right), \quad \nabla f(\mathbf{x}) := \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix} = Df_{\mathbf{x}}^\top.$$

Let f be **twice continuously differentiable** at $\mathbf{x}_0$, then

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

Where do we find a direction of **descent**?

Method of steepest descent

Taylor expansion at $\mathbf{x}_0$:

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

Method of steepest descent

Taylor expansion at $\mathbf{x}_0$:

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

Cauchy–Schwarz inequality:

$$-\|\mathbf{u}\|_2\|\mathbf{v}\|_2 \leq \langle \mathbf{u}, \mathbf{v} \rangle \leq \|\mathbf{u}\|_2\|\mathbf{v}\|_2, \quad (="", \text{ iff } \mathbf{u}, \mathbf{v} \text{ parallel}).$$

Method of steepest descent

Taylor expansion at $\mathbf{x}_0$:

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

Cauchy–Schwarz inequality:

$$-\|\mathbf{u}\|_2\|\mathbf{v}\|_2 \leq \langle \mathbf{u}, \mathbf{v} \rangle \leq \|\mathbf{u}\|_2\|\mathbf{v}\|_2, \quad (="", \text{ iff } \mathbf{u}, \mathbf{v} \text{ parallel}).$$

Choose $\mathbf{x}_1$ with $\mathbf{x}_1 - \mathbf{x}_0 = -\nabla f(\mathbf{x}_0)\alpha$, i.e., $\mathbf{x}_1 = \mathbf{x}_0 - \nabla f(\mathbf{x}_0)\alpha$,

$$f(\mathbf{x}_1) = f(\mathbf{x}_0) - \alpha\|\nabla f(\mathbf{x}_0)\|_2^2 + O(\alpha^2\|\nabla f(\mathbf{x}_0)\|_2^2),$$

for suitable step length $\alpha > 0$ decrease. Negative gradient is direction of steepest descent.

Method of steepest descent

Taylor expansion at $\mathbf{x}_0$:

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

Cauchy–Schwarz inequality:

$$-\|\mathbf{u}\|_2\|\mathbf{v}\|_2 \leq \langle \mathbf{u}, \mathbf{v} \rangle \leq \|\mathbf{u}\|_2\|\mathbf{v}\|_2, \quad (="", \text{ iff } \mathbf{u}, \mathbf{v} \text{ parallel}).$$

Choose $\mathbf{x}_1$ with $\mathbf{x}_1 - \mathbf{x}_0 = -\nabla f(\mathbf{x}_0)\alpha$, i.e., $\mathbf{x}_1 = \mathbf{x}_0 - \nabla f(\mathbf{x}_0)\alpha$,

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \alpha_k \|\nabla f(\mathbf{x}_k)\|_2^2 + O(\alpha_k^2 \|\nabla f(\mathbf{x}_k)\|_2^2),$$

for suitable step length $\alpha_k > 0$ decrease. Negative gradient is direction of steepest descent.

Method of steepest descent

Taylor expansion at $\mathbf{x}_0$:

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

Cauchy–Schwarz inequality:

$$-\|\mathbf{u}\|_2\|\mathbf{v}\|_2 \leq \langle \mathbf{u}, \mathbf{v} \rangle \leq \|\mathbf{u}\|_2\|\mathbf{v}\|_2, \quad (="", \text{ iff } \mathbf{u}, \mathbf{v} \text{ parallel}).$$

Choose $\mathbf{x}_1$ with $\mathbf{x}_1 - \mathbf{x}_0 = -\nabla f(\mathbf{x}_0)\alpha$, i.e., $\mathbf{x}_1 = \mathbf{x}_0 - \nabla f(\mathbf{x}_0)\alpha$,

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \alpha_k \|\nabla f(\mathbf{x}_k)\|_2^2 + O(\alpha_k^2 \|\nabla f(\mathbf{x}_k)\|_2^2),$$

for suitable step length $\alpha_k > 0$ decrease. **Negative gradient** is **direction of steepest descent**.

Variants: directions of descent that point more or less in direction of the negative gradient, step length $\alpha_k > 0$ variable.

Method of steepest descent

Expansion

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \alpha_k \|\nabla f(\mathbf{x}_k)\|_2^2 + O(\alpha_k^2 \|\nabla f(\mathbf{x}_k)\|_2^2)$$

indicates:

- ▶ $\alpha = \alpha_k$ **should be small**, otherwise mostly no improvement
- ▶ $\alpha = \alpha_k$ **too small**, improvement possibly very small

Method of steepest descent

Expansion

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \alpha_k \|\nabla f(\mathbf{x}_k)\|_2^2 + O(\alpha_k^2 \|\nabla f(\mathbf{x}_k)\|_2^2)$$

indicates:

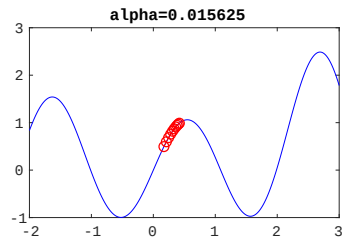
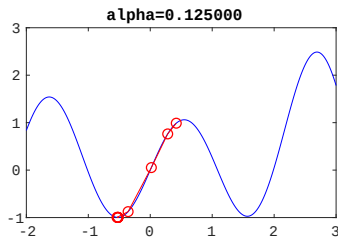
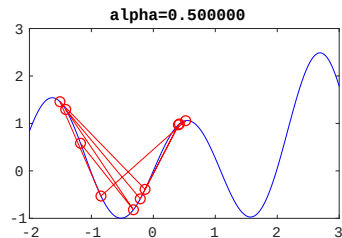
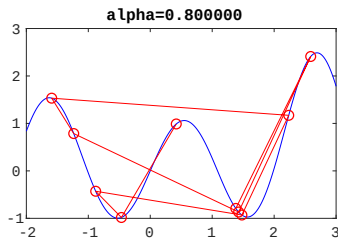
- ▶ $\alpha = \alpha_k$ **should be small**, otherwise mostly no improvement
- ▶ $\alpha = \alpha_k$ **too small**, improvement possibly very small

Next plot: **Method of steepest descent**

- ▶ 10 steps
- ▶ fixed step length α
- ▶ start value $x_0 = 0.42$

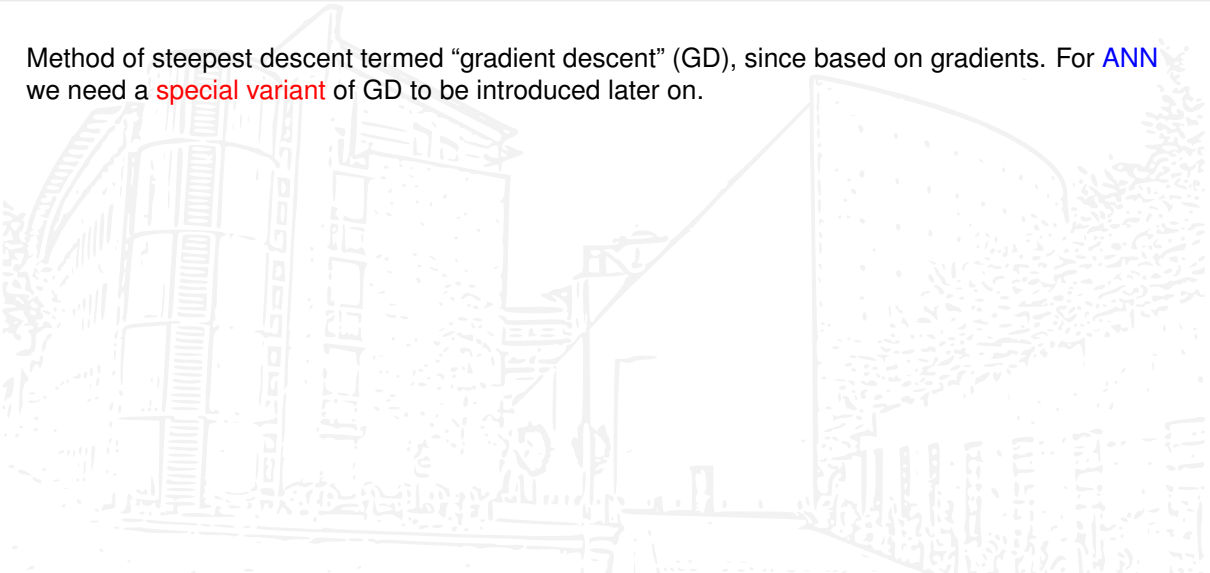
A step length around $\alpha = 1/8$ seems to be **optimal** in this example.

Method of steepest descent



Method of steepest descent

Method of steepest descent termed “gradient descent” (GD), since based on gradients. For ANN we need a **special variant** of GD to be introduced later on.



Method of steepest descent

Method of steepest descent termed “gradient descent” (GD), since based on gradients. For ANN we need a **special variant** of GD to be introduced later on.

The **optimal choice of α** depends on local behaviour of the function, i.e., the function value $f(\mathbf{x}_0)$, the first derivative, e.g., the gradient $\nabla f(\mathbf{x}_0)$, and the second derivative, the so-called **Hessian $\nabla^2 f(\mathbf{x}_0)$** (e.g., estimated by norm).

Method of steepest descent

Method of steepest descent termed “gradient descent” (GD), since based on gradients. For ANN we need a **special variant** of GD to be introduced later on.

The **optimal choice of α** depends on local behaviour of the function, i.e., the function value $f(\mathbf{x}_0)$, the first derivative, e.g., the gradient $\nabla f(\mathbf{x}_0)$, and the second derivative, the so-called **Hessian $\nabla^2 f(\mathbf{x}_0)$** (e.g., estimated by norm).

The iterates of GD change when **transformed linearly**,

$$\begin{aligned} f(\mathbf{x}) = f(\mathbf{A}^\top \mathbf{y}) = g(\mathbf{y}), \quad \mathbf{A}^\top \mathbf{y} = \mathbf{x} &\Rightarrow \mathbf{y}_{k+1} = \mathbf{y}_k - \nabla_{\mathbf{y}} g(\mathbf{y}_k) \alpha_k = \mathbf{y}_k - \mathbf{A} \nabla_{\mathbf{x}} f(\mathbf{x}_k) \alpha_k \\ &\Rightarrow \mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{A}^\top \mathbf{A} \nabla_{\mathbf{x}} f(\mathbf{x}_k) \alpha_k, \end{aligned}$$

therefore we can multiply the gradient with **arbitrarily chosen positive definite matrices**.

Newton's method

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ **twice continuously differentiable**, $\mathbf{x}$ simple minimum:

- ▶ **gradient equal to zero**: $\nabla f(\mathbf{x}) = \mathbf{0}$
- ▶ Hessian $\nabla^2 f(\mathbf{x})$ positive definite

Newton's method

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ **twice continuously differentiable**, $\mathbf{x}$ simple minimum:

- ▶ **gradient equal to zero**: $\nabla f(\mathbf{x}) = \mathbf{0}$
- ▶ Hessian $\nabla^2 f(\mathbf{x})$ positive definite

Newton's method to find a **stationary point** $\nabla f(\mathbf{x}) = \mathbf{0}$: $f \in C^3(\mathbb{R}^n)$, locally

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^\top \nabla^2 f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + O(\|\mathbf{x} - \mathbf{x}_0\|_2^3),$$

Newton's method

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ **twice continuously differentiable**, $\mathbf{x}$ simple minimum:

- ▶ **gradient equal to zero**: $\nabla f(\mathbf{x}) = \mathbf{0}$
- ▶ Hessian $\nabla^2 f(\mathbf{x})$ positive definite

Newton's method to find a **stationary point** $\nabla f(\mathbf{x}) = \mathbf{0}$: $f \in C^3(\mathbb{R}^n)$, locally

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^\top \nabla^2 f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + O(\|\mathbf{x} - \mathbf{x}_0\|_2^3),$$

i.e., **derived**

$$\nabla f(\mathbf{x}) = \nabla f(\mathbf{x}_0) + \nabla^2 f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

Newton's method

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ **twice continuously differentiable**, $\mathbf{x}$ simple minimum:

- ▶ **gradient equal to zero**: $\nabla f(\mathbf{x}) = \mathbf{0}$
- ▶ Hessian $\nabla^2 f(\mathbf{x})$ positive definite

Newton's method to find a **stationary point** $\nabla f(\mathbf{x}) = \mathbf{0}$: $f \in C^3(\mathbb{R}^n)$, locally

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^T \nabla^2 f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + O(\|\mathbf{x} - \mathbf{x}_0\|_2^3),$$

i.e., **derived**

$$\nabla f(\mathbf{x}) = \nabla f(\mathbf{x}_0) + \nabla^2 f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

Root of the first derivative up to $O(\|\mathbf{x}_1 - \mathbf{x}_0\|_2^2)$

$$\mathbf{x}_1 := \mathbf{x}_0 - (\nabla^2 f(\mathbf{x}_0))^{-1} \nabla f(\mathbf{x}_0).$$

Newton's method

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ **twice continuously differentiable**, $\mathbf{x}$ simple minimum:

- ▶ **gradient equal to zero**: $\nabla f(\mathbf{x}) = \mathbf{0}$
- ▶ Hessian $\nabla^2 f(\mathbf{x})$ positive definite

Newton's method to find a **stationary point** $\nabla f(\mathbf{x}) = \mathbf{0}$: $f \in C^3(\mathbb{R}^n)$, locally

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^T \nabla^2 f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + O(\|\mathbf{x} - \mathbf{x}_0\|_2^3),$$

i.e., **derived**

$$\nabla f(\mathbf{x}) = \nabla f(\mathbf{x}_0) + \nabla^2 f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2).$$

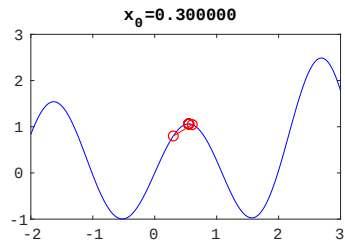
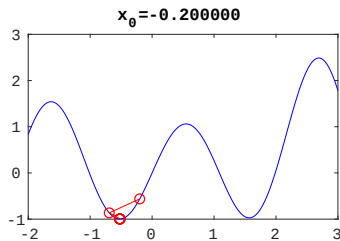
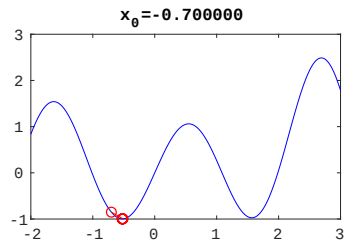
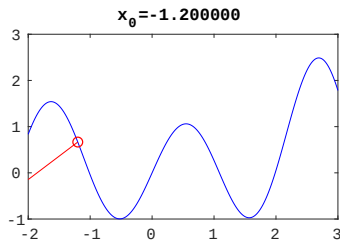
Root of the first derivative up to $O(\|\mathbf{x}_1 - \mathbf{x}_0\|_2^2)$

$$\mathbf{x}_1 := \mathbf{x}_0 - (\nabla^2 f(\mathbf{x}_0))^{-1} \nabla f(\mathbf{x}_0).$$

Adding **step length and line search**:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - (\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k) \alpha_k, \quad k \in \mathbb{N}_0.$$

Newton's method



Newton's method

Depicted: Newton's method with $\alpha_k = 1$ for 10 steps.



Newton's method

Depicted: Newton's method with $\alpha_k = 1$ for 10 steps.

Remarks:

- ▶ Newton's method converges **locally quadratic**
- ▶ it can **jump wildly around** if far from solution
- ▶ if Hessian $\nabla^2 f(\mathbf{x}_k)$ is **SPD** then Newton's method is a **variant of GD**

Newton's method

Depicted: Newton's method with $\alpha_k = 1$ for 10 steps.

Remarks:

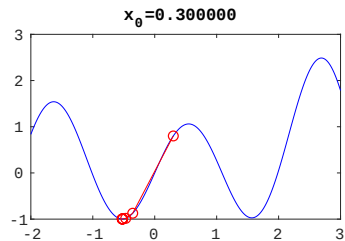
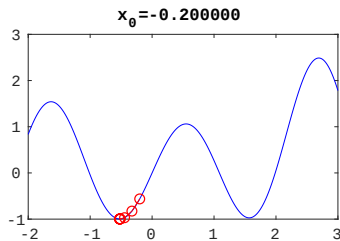
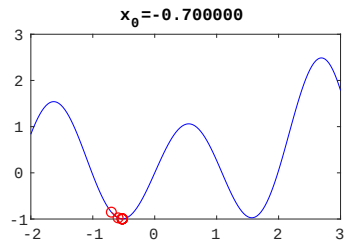
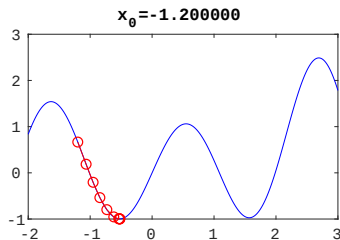
- ▶ Newton's method converges **locally quadratic**
- ▶ it can **jump wildly around** if far from solution
- ▶ if Hessian $\nabla^2 f(\mathbf{x}_k)$ is **SPD** then Newton's method is a **variant of GD**

Modification, e.g.:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - (\nabla^2 f(\mathbf{x}_k) + c \|\nabla f(\mathbf{x}_k)\|_2^2 \mathbf{I})^{-1} \nabla f(\mathbf{x}_k) \alpha_k, \quad k \in \mathbb{N}_0, \quad c > 0.$$

Next picture: with $c = 2.4$ modified Newton's method with $\alpha = 1$ for 10 steps.

Newton's method



Stochastic gradient descent

GD (“Batch Gradient Descent”): computation of $\nabla C(\mathbf{p})$, update by

$$\mathbf{p} \leftarrow \mathbf{p} - \nabla C(\mathbf{p})\eta = \mathbf{p} - \frac{1}{N} \sum_{j=1}^N \nabla C_j(\mathbf{p})\eta.$$

Step length $\eta > 0$ in ML: “learning rate”.

Stochastic gradient descent

GD (“Batch Gradient Descent”): computation of $\nabla C(\mathbf{p})$, update by

$$\mathbf{p} \leftarrow \mathbf{p} - \nabla C(\mathbf{p})\eta = \mathbf{p} - \frac{1}{N} \sum_{j=1}^N \nabla C_j(\mathbf{p})\eta.$$

Step length $\eta > 0$ in ML: “**learning rate**”.

Stochastic gradient descent (SGD): one **epoch** given by

compute permutation σ of $\{1, \dots, N\}$, $\mathbf{p} \leftarrow \mathbf{p} - \nabla C_{\sigma(j)}(\mathbf{p})\eta, \quad j = 1, \dots, N.$

Stochastic gradient descent

GD (“Batch Gradient Descent”): computation of $\nabla C(\mathbf{p})$, update by

$$\mathbf{p} \leftarrow \mathbf{p} - \nabla C(\mathbf{p})\eta = \mathbf{p} - \frac{1}{N} \sum_{j=1}^N \nabla C_j(\mathbf{p})\eta.$$

Step length $\eta > 0$ in ML: “**learning rate**”.

Stochastic gradient descent (SGD): one **epoch** given by

compute permutation σ of $\{1, \dots, N\}$, $\mathbf{p} \leftarrow \mathbf{p} - \nabla C_{\sigma(j)}(\mathbf{p})\eta$, $j = 1, \dots, N$.

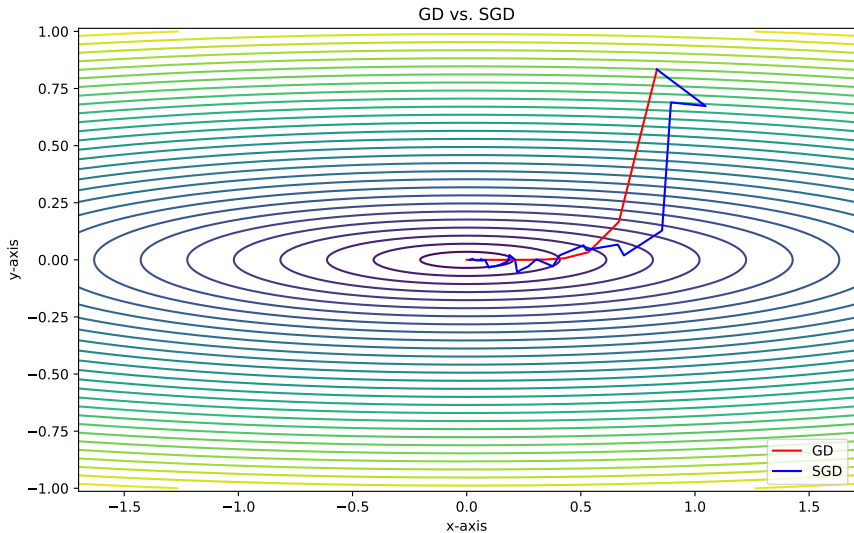
SGD w/ minibatches: mean of k partial derivatives, “better” direction of descent, $k \ll N$,

$$\nabla C_{j:j+k-1}(\mathbf{p}) := \frac{1}{k} \sum_{\ell=j}^{j+k-1} \nabla C_{\ell}(\mathbf{p}),$$

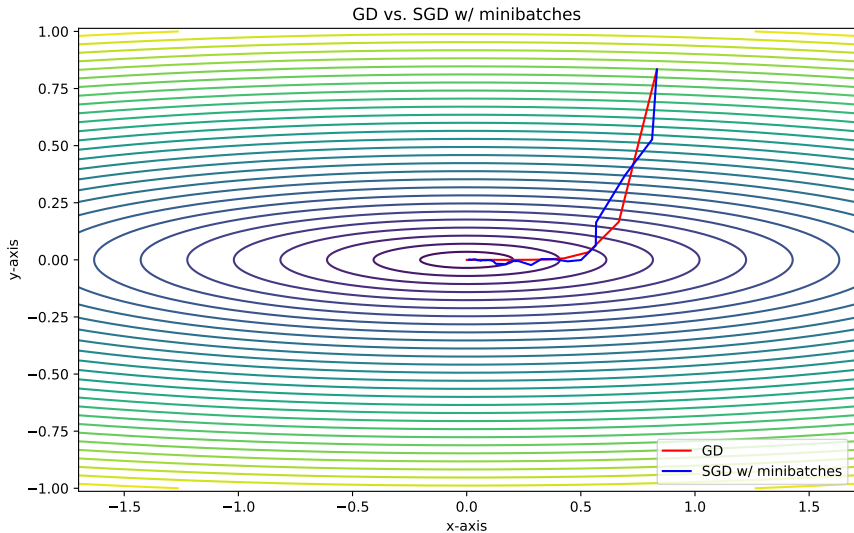
minibatch update ($k = 1$ SGD, $k = N$ classical GD; one pass = epoch):

$$\mathbf{p} \leftarrow \mathbf{p} - \nabla C_{\sigma(j:\min(j+k-1, N))}(\mathbf{p})\eta, \quad j = 1, 1+k, 1+2k, \dots, N.$$

Stochastic gradient descent



Stochastic gradient descent



Multidimensional chain rule

SGD (with minibatches) for ANN: derivative of

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m?$$

Multidimensional chain rule

SGD (with minibatches) for ANN: derivative of

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := \sigma(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m?$$

Multidimensional chain rule:

$$\mathbf{h}(\mathbf{x}) := \mathbf{f}(\mathbf{g}(\mathbf{x})), \quad \mathbf{y} := \mathbf{g}(\mathbf{x}) \quad \Rightarrow \quad \mathbf{J}_{\mathbf{h}}(\mathbf{x}) = \mathbf{J}_{\mathbf{f}}(\mathbf{y}) \mathbf{J}_{\mathbf{g}}(\mathbf{x}), \quad \mathbf{J}_{\mathbf{h}} \in \mathbb{R}^{m \times n}, \quad \mathbf{J}_{\mathbf{f}} \in \mathbb{R}^{m \times k}, \quad \mathbf{J}_{\mathbf{g}} \in \mathbb{R}^{k \times n}.$$

Multidimensional chain rule

SGD (with minibatches) for ANN: derivative of

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := \sigma(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m?$$

Multidimensional chain rule:

$$\mathbf{h}(\mathbf{x}) := \mathbf{f}(\mathbf{g}(\mathbf{x})), \quad \mathbf{y} := \mathbf{g}(\mathbf{x}) \quad \Rightarrow \quad \mathbf{J}_h(\mathbf{x}) = \mathbf{J}_f(\mathbf{y}) \mathbf{J}_g(\mathbf{x}), \quad \mathbf{J}_h \in \mathbb{R}^{m \times n}, \quad \mathbf{J}_f \in \mathbb{R}^{m \times k}, \quad \mathbf{J}_g \in \mathbb{R}^{k \times n}.$$

Example of Jacobian $\mathbf{J}_g \in \mathbb{R}^{k \times n}$:

$$\mathbf{J}_g(\mathbf{x}) := D\mathbf{g}(\mathbf{x}) = \begin{pmatrix} \frac{\partial g_1}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_1}{\partial x_n}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial g_k}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_k}{\partial x_n}(\mathbf{x}) \end{pmatrix}.$$

Multidimensional chain rule

SGD (with minibatches) for ANN: derivative of

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := \sigma(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m?$$

Multidimensional chain rule:

$$\mathbf{h}(\mathbf{x}) := \mathbf{f}(\mathbf{g}(\mathbf{x})), \quad \mathbf{y} := \mathbf{g}(\mathbf{x}) \quad \Rightarrow \quad \mathbf{J}_h(\mathbf{x}) = \mathbf{J}_f(\mathbf{y}) \mathbf{J}_g(\mathbf{x}), \quad \mathbf{J}_h \in \mathbb{R}^{m \times n}, \quad \mathbf{J}_f \in \mathbb{R}^{m \times k}, \quad \mathbf{J}_g \in \mathbb{R}^{k \times n}.$$

Example of Jacobian $\mathbf{J}_g \in \mathbb{R}^{k \times n}$:

$$\mathbf{J}_g(\mathbf{x}) := D\mathbf{g}(\mathbf{x}) = \begin{pmatrix} \frac{\partial g_1}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_1}{\partial x_n}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial g_k}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_k}{\partial x_n}(\mathbf{x}) \end{pmatrix}.$$

Transposed derivative for gradients $\rightsquigarrow$ order of matrix products in chain rule reversed.

Backpropagation

Theorem (Partial derivatives of ANN, part 1/2)

ANN & local cost function given,

$$C(\mathbf{p}) := \frac{1}{2} \|\mathbf{a}_m - \mathbf{y}\|_2^2, \quad \mathbf{a}_m = \mathbf{f}(\mathbf{x}).$$

Define for $i = 1, \dots, m-1$ the vectors

$$\boldsymbol{\delta}_i := \frac{\partial C}{\partial \mathbf{z}_i} := \begin{pmatrix} \frac{\partial C}{\partial z_{1,i}} \\ \vdots \\ \frac{\partial C}{\partial z_{n_{i+1},i}} \end{pmatrix} = \nabla_{\mathbf{z}_i} C.$$

Vectors $\boldsymbol{\delta}_i$ can be computed recursively *from last layer to first*,

$$\boldsymbol{\delta}_{m-1} = a'_{m-1}(\mathbf{z}_{m-1}) \circ (\mathbf{a}_m - \mathbf{y}), \quad \boldsymbol{\delta}_i = a'_i(\mathbf{z}_i) \circ (\mathbf{W}_{i+1}^\top \boldsymbol{\delta}_{i+1}), \quad i = m-2, m-3, \dots, 1.$$

Backpropagation

Theorem (Partial derivatives of ANN, part 2/2)

Derivatives of C with respect to the weights $\mathbf{W}_i$ and the biases $\mathbf{b}_i$, $i = 1, \dots, m - 1$, are given by

$$\frac{\partial C}{\partial \mathbf{W}_i} = \delta_i \mathbf{a}_i^\top, \quad \frac{\partial C}{\partial \mathbf{b}_i} = \delta_i.$$

Backpropagation

Theorem (Partial derivatives of ANN, part 2/2)

Derivatives of C with respect to the weights $\mathbf{W}_i$ and the biases $\mathbf{b}_i$, $i = 1, \dots, m - 1$, are given by

$$\frac{\partial C}{\partial \mathbf{W}_i} = \delta_i \mathbf{a}_i^\top, \quad \frac{\partial C}{\partial \mathbf{b}_i} = \delta_i.$$

Proof. (Part 1)

The derivative of the partial cost function with respect to $\mathbf{a}_m$, interpreted as a column vector, i.e., gradient, is given by

$$\frac{\partial C}{\partial \mathbf{a}_m} = \frac{1}{2} \frac{\partial \langle \mathbf{a}_m - \mathbf{y}, \mathbf{a}_m - \mathbf{y} \rangle}{\partial \mathbf{a}_m} = \frac{1}{2} \frac{\partial (\mathbf{a}_m^\top \mathbf{a}_m - 2\mathbf{y}^\top \mathbf{a}_m + \mathbf{y}^\top \mathbf{y})}{\partial \mathbf{a}_m} = \mathbf{a}_m - \mathbf{y} = \nabla_{\mathbf{a}_m} C.$$

Backpropagation

Proof. (Part 2)

For $i = 1, \dots, m - 1$, since $\mathbf{a}_{i+1} = a_i(\mathbf{z}_i)$, $\mathbf{z}_i \in \mathbb{R}^{n_{i+1}}$,

$$\frac{\partial \mathbf{a}_{i+1}}{\partial \mathbf{z}_i} := (\mathbf{J}_{\mathbf{a}_{i+1}}(\mathbf{z}_i))^T = \begin{pmatrix} \frac{\partial a_i(z_1^{(i)})}{\partial z_1^{(i)}} & \dots & \frac{\partial a_i(z_{n_{i+1}}^{(i)})}{\partial z_1^{(i)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial a_i(z_1^{(i)})}{\partial z_{n_{i+1}}^{(i)}} & \dots & \frac{\partial a_i(z_{n_{i+1}}^{(i)})}{\partial z_{n_{i+1}}^{(i)}} \end{pmatrix} = \text{diag}(a'_i(\mathbf{z}_i)).$$

Backpropagation

Proof. (Part 2)

For $i = 1, \dots, m - 1$, since $\mathbf{a}_{i+1} = a_i(\mathbf{z}_i)$, $\mathbf{z}_i \in \mathbb{R}^{n_{i+1}}$,

$$\frac{\partial \mathbf{a}_{i+1}}{\partial \mathbf{z}_i} := (\mathbf{J}_{\mathbf{a}_{i+1}}(\mathbf{z}_i))^T = \begin{pmatrix} \frac{\partial a_i(z_1^{(i)})}{\partial z_1^{(i)}} & \dots & \frac{\partial a_i(z_{n_{i+1}}^{(i)})}{\partial z_1^{(i)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial a_i(z_1^{(i)})}{\partial z_{n_{i+1}}^{(i)}} & \dots & \frac{\partial a_i(z_{n_{i+1}}^{(i)})}{\partial z_{n_{i+1}}^{(i)}} \end{pmatrix} = \text{diag}(a'_i(\mathbf{z}_i)).$$

Due to the (transposed) chain rule,

$$\delta_{m-1} = \frac{\partial C}{\partial \mathbf{z}_{m-1}} = \frac{\partial \mathbf{a}_m}{\partial \mathbf{z}_{m-1}} \frac{\partial C}{\partial \mathbf{a}_m} = \text{diag}(a'_{m-1}(\mathbf{z}_{m-1}))(\mathbf{a}_m - \mathbf{y}) = a'_{m-1}(\mathbf{z}_{m-1}) \circ (\mathbf{a}_m - \mathbf{y}),$$

where $\circ$ denotes the **componentwise multiplication** or **Hadamard product**.

Backpropagation

Proof. (Part 3)

With $\mathbf{z}_{i+1} = \mathbf{W}_{i+1}\mathbf{a}_{i+1} + \mathbf{b}_{i+1}$ for $i = 0, \dots, m-2$ and thus

$$\frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{a}_{i+1}} = (\mathbf{J}_{\mathbf{z}_{i+1}}(\mathbf{a}_{i+1}))^\top = \mathbf{W}_{i+1}^\top,$$

Backpropagation

Proof. (Part 3)

With $\mathbf{z}_{i+1} = \mathbf{W}_{i+1}\mathbf{a}_{i+1} + \mathbf{b}_{i+1}$ for $i = 0, \dots, m-2$ and thus

$$\frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{a}_{i+1}} = (\mathbf{J}_{\mathbf{z}_{i+1}}(\mathbf{a}_{i+1}))^\top = \mathbf{W}_{i+1}^\top,$$

we conclude, again using the (transposed) chain rule, that

$$\delta_i = \frac{\partial C}{\partial \mathbf{z}_i} = \frac{\partial \mathbf{a}_{i+1}}{\partial \mathbf{z}_i} \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{a}_{i+1}} \frac{\partial C}{\partial \mathbf{z}_{i+1}} = \text{diag}(a'_i(\mathbf{z}_i)) \mathbf{W}_{i+1}^\top \delta_{i+1} = a'_i(\mathbf{z}_i) \circ (\mathbf{W}_{i+1}^\top \delta_{i+1})$$

for all $i = 1, \dots, m-2$.

Backpropagation

Proof. (Part 3)

With $\mathbf{z}_{i+1} = \mathbf{W}_{i+1}\mathbf{a}_{i+1} + \mathbf{b}_{i+1}$ for $i = 0, \dots, m-2$ and thus

$$\frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{a}_{i+1}} = (\mathbf{J}_{\mathbf{z}_{i+1}}(\mathbf{a}_{i+1}))^\top = \mathbf{W}_{i+1}^\top,$$

we conclude, again using the (transposed) chain rule, that

$$\delta_i = \frac{\partial C}{\partial \mathbf{z}_i} = \frac{\partial \mathbf{a}_{i+1}}{\partial \mathbf{z}_i} \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{a}_{i+1}} \frac{\partial C}{\partial \mathbf{z}_{i+1}} = \text{diag}(a'_i(\mathbf{z}_i)) \mathbf{W}_{i+1}^\top \delta_{i+1} = a'_i(\mathbf{z}_i) \circ (\mathbf{W}_{i+1}^\top \delta_{i+1})$$

for all $i = 1, \dots, m-2$. The equations for the derivative of C with respect to $\mathbf{b}_i$ follow again using the (transposed) chain rule, since $\mathbf{z}_i = \mathbf{W}_i\mathbf{a}_i + \mathbf{b}_i$ and

$$\frac{\partial \mathbf{z}_i}{\partial \mathbf{b}_i} = (\mathbf{J}_{\mathbf{z}_i}(\mathbf{b}_i))^\top = \mathbf{I}, \quad \text{i.e.,} \quad \frac{\partial C}{\partial \mathbf{b}_i} = \frac{\partial \mathbf{z}_i}{\partial \mathbf{b}_i} \frac{\partial C}{\partial \mathbf{z}_i} = \mathbf{I} \delta_i = \delta_i.$$

Backpropagation

Last part of the proof.

To compute $\partial C / \partial \mathbf{W}_i$ we drop the indices, i.e., we use $\delta = \partial C / \partial \mathbf{z}$ and $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$. The derivative of $z_i = \mathbf{e}_i^\top \mathbf{z}$ with respect to $w_{i,j} = \mathbf{e}_i^\top \mathbf{W} \mathbf{e}_j$ is given by

$$\frac{\partial z_i}{\partial w_{i,j}} = \frac{\partial (\sum_k w_{i,k} a_k + b_i)}{\partial w_{i,j}} = a_j = \mathbf{a}^\top \mathbf{e}_j,$$

the other derivatives $\partial z_\ell / \partial w_{i,j}$ for $\ell \neq i$ are all zero.



Backpropagation

Last part of the proof.

To compute $\partial C / \partial \mathbf{W}_i$ we drop the indices, i.e., we use $\delta = \partial C / \partial \mathbf{z}$ and $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$. The derivative of $z_i = \mathbf{e}_i^\top \mathbf{z}$ with respect to $w_{i,j} = \mathbf{e}_i^\top \mathbf{W} \mathbf{e}_j$ is given by

$$\frac{\partial z_i}{\partial w_{i,j}} = \frac{\partial (\sum_k w_{i,k} a_k + b_i)}{\partial w_{i,j}} = a_j = \mathbf{a}^\top \mathbf{e}_j,$$

the other derivatives $\partial z_\ell / \partial w_{i,j}$ for $\ell \neq i$ are all zero.

Therefore the derivative of C with respect to $w_{i,j}$ and the “matrix derivative” $\partial C / \partial \mathbf{W}$ are given by the one-dimensional chain rule as

$$\frac{\partial C}{\partial w_{i,j}} = \frac{\partial C}{\partial z_i} \frac{\partial z_i}{\partial w_{i,j}} = \delta_i a_j = \mathbf{e}_i^\top \boldsymbol{\delta} \mathbf{a}^\top \mathbf{e}_j \quad \Rightarrow \quad \frac{\partial C}{\partial \mathbf{W}} = \boldsymbol{\delta} \mathbf{a}^\top.$$

Substituting the missing layer index i finishes the proof. □

Backpropagation

Backpropagation: algorithm for the computation of derivatives and better parameters $\mathbf{p}$ (weights $\mathbf{W}_i$ and biases $\mathbf{b}_i, i = 1, \dots, m - 1$), termed “backpropagation” since based on **backward loop** over layers that propagates output errors through all layers.

Backpropagation

Backpropagation: algorithm for the computation of derivatives and better parameters $\mathbf{p}$ (weights $\mathbf{W}_i$ and biases $\mathbf{b}_i$, $i = 1, \dots, m - 1$), termed “backpropagation” since based on **backward loop** over layers that propagates output errors through all layers.

Implementation of minibatches:

- ▶ **simultaneous evaluation** $\rightsquigarrow$ matrix-matrix multiplications $\mathbf{Z}_i = \mathbf{W}_i \mathbf{A}_i + \mathbf{b}_i \mathbf{e}^\top$, initialization $\mathbf{A}_1 = (\mathbf{x}_1, \dots, \mathbf{x}_k)$, $\mathbf{e} \in \mathbb{R}^k$ vector comprising ones
- ▶ evaluation of **activation functions elementwise**, $\mathbf{A}_{i+1} = a_i(\mathbf{Z}_i)$
- ▶ **backpropagation with matrices** $\rightsquigarrow$ derivative of errors $\mathbf{A}_m - (\mathbf{y}_1, \dots, \mathbf{y}_k)$, all $\delta_i^{(1)}, \dots, \delta_i^{(k)}$ terms simultaneously using matrix-matrix multiplications as matrices Δ_i
- ▶ **averaged updates** are $\Delta_i \mathbf{A}_i^\top / k$ and $\Delta_i \mathbf{e} / k$

Backpropagation

Backpropagation: algorithm for the computation of derivatives and better parameters $\mathbf{p}$ (weights $\mathbf{W}_i$ and biases $\mathbf{b}_i$, $i = 1, \dots, m - 1$), termed “backpropagation” since based on **backward loop** over layers that propagates output errors through all layers.

Implementation of minibatches:

- ▶ **simultaneous evaluation** $\rightsquigarrow$ matrix-matrix multiplications $\mathbf{Z}_i = \mathbf{W}_i \mathbf{A}_i + \mathbf{b}_i \mathbf{e}^\top$, initialization $\mathbf{A}_1 = (\mathbf{x}_1, \dots, \mathbf{x}_k)$, $\mathbf{e} \in \mathbb{R}^k$ vector comprising ones
- ▶ evaluation of **activation functions elementwise**, $\mathbf{A}_{i+1} = a_i(\mathbf{Z}_i)$
- ▶ **backpropagation with matrices** $\rightsquigarrow$ derivative of errors $\mathbf{A}_m - (\mathbf{y}_1, \dots, \mathbf{y}_k)$, all $\delta_i^{(1)}, \dots, \delta_i^{(k)}$ terms simultaneously using matrix-matrix multiplications as matrices Δ_i
- ▶ **averaged updates** are $\Delta_i \mathbf{A}_i^\top / k$ and $\Delta_i \mathbf{e} / k$
- ▶ **Better**: backpropagation of **averaged error** $(\mathbf{A}_m - (\mathbf{y}_1, \dots, \mathbf{y}_k)) / k$, only one initial division by k

References I

- [1] Yann Le Cun. “Modèles connexionnistes de l'apprentissage”. Thèse doctorat. Université Pierre-et-Marie-Curie, June 1987. URL: <http://yann.lecun.com/exdb/publis/psgz/lecun-87.ps.gz>.
- [2] G. Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Math. Control Signals Systems* 2.4 (1989), pp. 303–314. ISSN: 0932-4194. DOI: 10.1007/BF02551274. URL: <https://doi.org/10.1007/BF02551274>.
- [3] Weinan E, Chao Ma, and Lei Wu. “A priori estimates of the population risk for two-layer neural networks”. In: *Comm. Math. Sci.* 17.5 (2019). Dedicated to the memory of Professor David Shen Ou Cai, pp. 1407–1425. DOI: 10.4310/CMS.2019.v17.n5.a11.
- [4] Weinan E, Chao Ma, and Lei Wu. “The Barron Space and the Flow-Induced Function Spaces for Neural Network Models”. In: *Constr. Approx.* (2021). DOI: 10.1007/s00365-021-09549-y.
- [5] Ken-ichi Funahashi. “On the Approximate Realization of Continuous Mappings by Neural Networks”. In: *Neural Networks* 2 (1989), pp. 183–192.

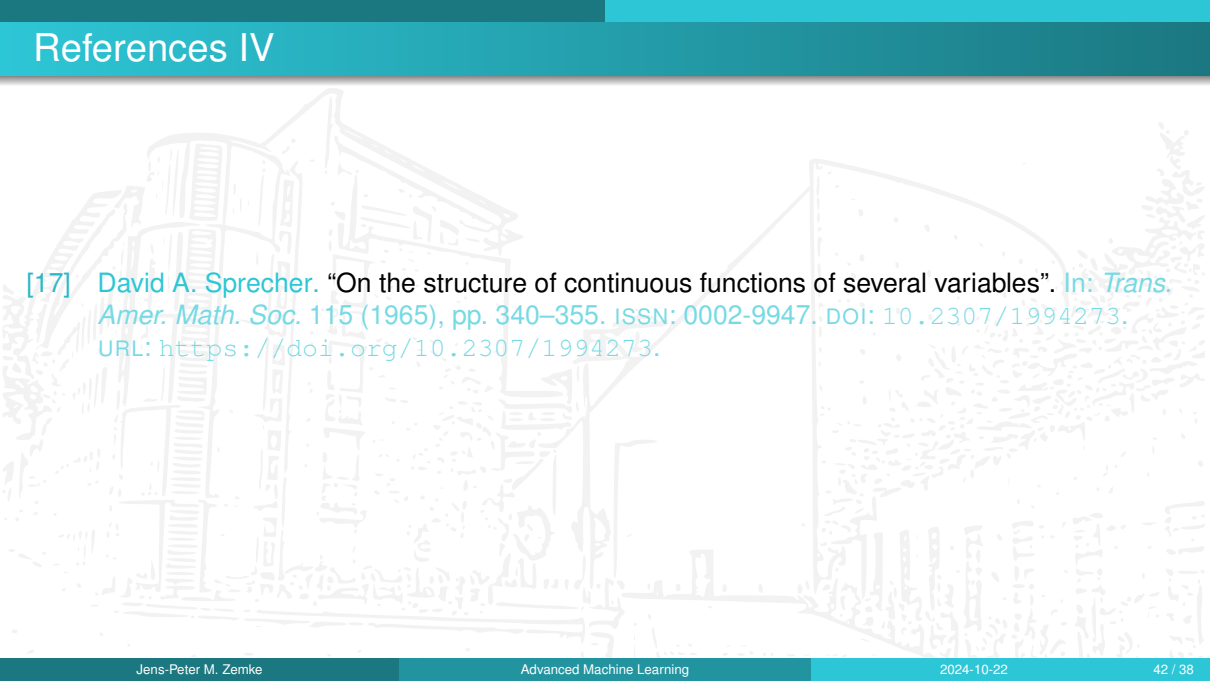
References II

- [6] Boris Hanin. *Universal Function Approximation by Deep Neural Nets with Bounded Width and ReLU Activations*. arXiv preprint arXiv:1708.02691v3 [stat.ML]. 2017. URL: <https://arxiv.org/abs/1708.02691v3>.
- [7] Kurt Hornik. “Approximation Capabilities of Multilayer Feedforward Networks”. In: *Neural Networks* 4 (1991), pp. 251–257.
- [8] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer Feedforward Networks are Universal Approximators”. In: *Neural Networks* 2 (1989), pp. 359–366.
- [9] A. N. Kolmogorov. “On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition”. In: *Dokl. Akad. Nauk SSSR* 114.5 (1957), pp. 953–956. ISSN: 0002-3264.
- [10] A. N. Kolmogorov. “On the representation of continuous functions of several variables by superpositions of continuous functions of a smaller number of variables”. In: *Amer. Math. Soc. Transl. (2)* 17 (1961), pp. 369–373. ISSN: 0065-9290. DOI: 10.1090/trans2/017/12. URL: <https://doi.org/10.1090/trans2/017/12>.

References III

- [11] Věra Kůrková. “Kolmogorov’s Theorem and Multilayer Neural Networks”. In: *Neural Networks* 5.3 (1992), pp. 501–506. DOI: 10.1016/0893-6080(92)90012-8.
- [12] Alan Lapedes and Robert Farber. “How neural nets work”. In: *NIPS’87 Proceedings of the 1987 International Conference on Neural Information Processing Systems*. MIT Press Cambridge, 1988, pp. 442–456. URL: <https://papers.nips.cc/paper/59-how-neural-nets-work.pdf>.
- [13] Moshe Leshno et al. “Multilayer Feedforward Networks With a Nonpolynomial Activation Function Can Approximate Any Function”. In: *Neural Networks* 6 (1993), pp. 861–867.
- [14] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. URL: <http://neuralnetworksanddeeplearning.com>.
- [15] J. Park and I. W. Sandberg. “Universal Approximation Using Radial-Basis-Function Networks”. In: *Neural Computation* 3 (1991), pp. 246–257.
- [16] Allan Pinkus. “Approximation theory of the MLP model in neural networks”. In: *Acta numerica*, 1999. Vol. 8. Acta Numer. Cambridge Univ. Press, Cambridge, 1999, pp. 143–195. DOI: 10.1017/S0962492900002919.

References IV

- 
- [17] David A. Sprecher. “On the structure of continuous functions of several variables”. In: *Trans. Amer. Math. Soc.* 115 (1965), pp. 340–355. ISSN: 0002-9947. DOI: 10.2307/1994273. URL: <https://doi.org/10.2307/1994273>.